

LAPORAN KUIS 2
MATA KULIAH BIG DATA



Dosen Pengampu:

M. Hasyim Ratsanjani, S.Kom., M.Kom.

Disusun Oleh:

Shasia Sasa Salsabyla	NIM. 2241720029
Sukma Bagus Wahasdwika	NIM. 2241720223
Triyana Dewi Fatmawati	NIM. 2241720206
Yuma Rakha Samodra Sikayo	NIM. 2241720194

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2025

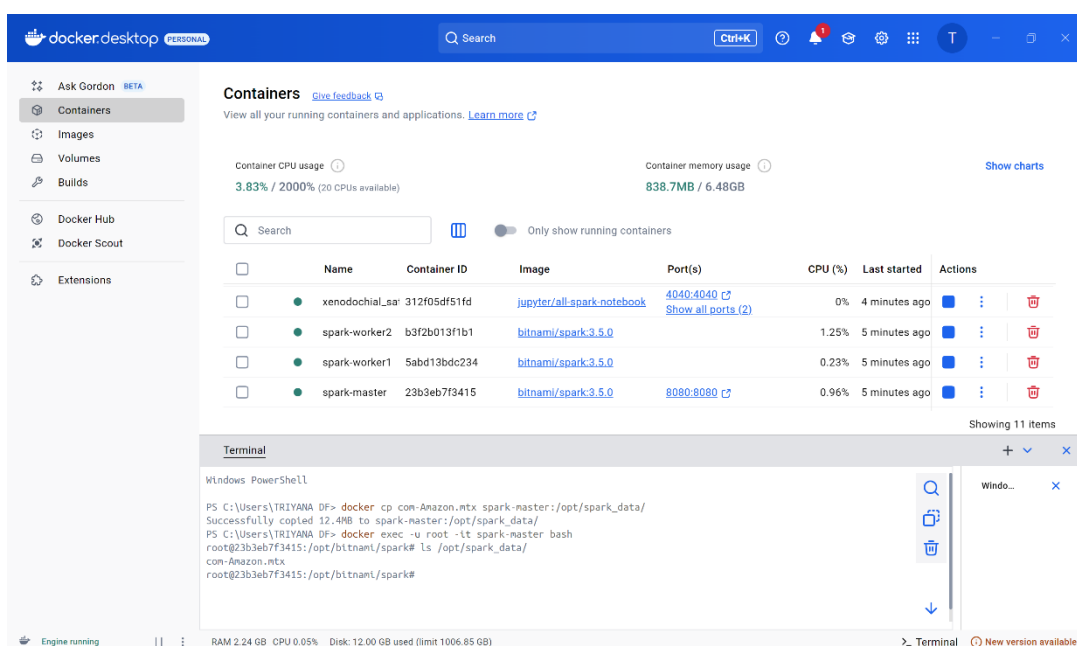
KUIS 2 Big Data: Analisis Pengaruh Produk Menggunakan Algoritma PageRank pada Jaringan Co-Purchasing Amazon

Soal atau Hasil yang Diharapkan :

Sebuah daftar produk (beserta ID-nya) yang diurutkan berdasarkan skor PageRank dari tertinggi ke terendah.

Pengerjaan :

Copy data ke /opt/spark_data/



The screenshot shows the Docker Desktop interface. The 'Containers' tab is active, displaying a list of running containers. Below the list, a terminal window is open, showing the execution of Docker commands to copy a file from the host to the 'spark-master' container and to enter the container's shell.

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
xenodochial_sar	312f05df51fd	jupyter/all-spark-notebook	4040:4040	0%	4 minutes ago	[Stop] [Restart] [Delete]
spark-worker2	b3f2b013f1b1	bitnami/spark-3.5.0		1.25%	5 minutes ago	[Stop] [Restart] [Delete]
spark-worker1	5abd13bdc234	bitnami/spark-3.5.0		0.23%	5 minutes ago	[Stop] [Restart] [Delete]
spark-master	23b3eb7f3415	bitnami/spark-3.5.0	8080:8080	0.96%	5 minutes ago	[Stop] [Restart] [Delete]

```
Windows PowerShell
PS C:\Users\TRIVANA DF> docker cp com-Amazon.mtx spark-master:/opt/spark_data/
Successfully copied 12.4kB to spark-master:/opt/spark_data/
PS C:\Users\TRIVANA DF> docker exec -u root -it spark-master bash
root@23b3eb7f3415:/opt/bitnami/spark# ls /opt/spark_data/
com-Amazon.mtx
root@23b3eb7f3415:/opt/bitnami/spark#
```

Yang dilakukan:

- Menyalin file com-Amazon.mtx dari direktori lokal ke dalam container spark-master.
- File ditempatkan di folder /opt/spark_data/ di dalam container.
- Perintah membuka terminal shell di dalam container spark-master sebagai user root.
- Mengecek apakah file com-Amazon.mtx berhasil masuk ke dalam container.

Output menunjukkan file tersebut tersedia. Artinya file sudah siap untuk dibaca oleh Spark.

1. Inisialisasi Spark Session dan GraphFrames

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from graphframes import GraphFrame # Library tambahan untuk graph processing di PySpark

# Inisialisasi Spark Session
spark = SparkSession.builder \
    .appName("Amazon PageRank") \
    .master("spark://spark-master:7077") \
    .config("spark.jars.packages", "graphframes:graphframes:0.8.2-spark3.2-s_2.12") \
    .getOrCreate()
```

Yang dilakukan:

- Import library PySpark dan GraphFrames

Fungsi:

- SparkSession adalah entry point utama untuk menjalankan Spark.
- functions dan types membantu saat memproses kolom DataFrame (misalnya split, cast, dsb).
- GraphFrame digunakan untuk membangun graf dan menjalankan algoritma PageRank di PySpark.

- Inisialisasi Spark Session

Penjelasan per baris:

- .appName("Amazon PageRank"): Menentukan nama aplikasi Spark, terlihat di UI Spark.
- .master("spark://spark-master:7077"): Menghubungkan Spark ke master node di cluster Docker kamu (sesuai setup default).
- .config("spark.jars.packages", ...): Mengunduh dan memuat library **GraphFrames** yang dibutuhkan untuk algoritma graf.
 - Versi 0.8.2-spark3.2-s_2.12 cocok untuk Spark 3.2 dan Scala 2.12.
- .getOrCreate(): Membuat session baru, atau mengambil session aktif jika sudah ada.

2. Membaca dan Membersihkan Data

```
# Baca file mtx
raw_rdd = spark.sparkContext.textFile("/opt/spark_data/com-Amazon.mtx")

# Filter baris komentar (%) dan baris pertama (dimensi matrix)
data_rdd = raw_rdd \
    .filter(lambda line: not line.startswith('%')) \
    .zipWithIndex() \
    .filter(lambda pair: pair[1] > 0) \
    .map(lambda pair: pair[0]) \
    .map(lambda line: tuple(line.strip().split())) \
    .filter(lambda x: x[0] != x[1]) # Hindari self-loop (opsional)

# Tampilkan 10 baris pertama
data_rdd.take(10)
```

[3]:

```
[('53526', '1'),
 ('71632', '1'),
 ('98006', '1'),
 ('148224', '1'),
 ('209320', '1'),
 ('268299', '1'),
 ('270060', '1'),
 ('302148', '1'),
 ('16283', '2'),
 ('37799', '2')]
```

Yang dilakukan:

- Membaca file .mtx

Fungsi:

- Membaca file com-Amazon.mtx sebagai RDD.
- Format Matrix Market .mtx biasanya berisi:
 - Beberapa baris awal berisi komentar (dimulai dengan %).
 - Satu baris berisi ukuran matriks (jumlah baris, kolom, dan edge).
 - Baris-baris selanjutnya berisi pasangan node (misal: 1 23 artinya node 1 terhubung ke node 23).

- Memfilter komentar dan baris ukuran

Penjelasan langkah-langkah:

- `.filter(lambda line: not line.startswith('%'))` : Hapus semua komentar (%).
- `.zipWithIndex()` : Tambahkan nomor indeks untuk melacak urutan baris.
- `.filter(lambda pair: pair[1] > 0)` : Buang baris **pertama non-komentar**, karena biasanya itu adalah metadata (misal: 334863 334863 925872) : jumlah node dan edge.

- `.map(lambda pair: pair[0])` : Ambil kembali teks barisnya tanpa indeks.
 - `.map(lambda line: tuple(line.strip().split()))` : Memecah string menjadi dua elemen (src dan dst) dan menjadikannya tuple seperti ('1', '23'). Contoh : baris 1 23 akan menjadi ('1', '23').
 - `.filter(lambda x: x[0] != x[1])` : Menghindari edge seperti ('42', '42'), yaitu hubungan dari produk ke dirinya sendiri (biasanya tidak relevan untuk PageRank).
- **Menampilkan 10 baris pertama**
- Berfungsi untuk **menampilkan 10 baris pertama** dari data_rdd, yaitu hasil preprocessing dari file .mtx.
 - Setiap tuple adalah pasangan produk yang dibeli bersamaan ((produk_sumber, produk_tujuan)).

3. Konversi ke DataFrame untuk GraphFrame

```
# Edges
edges_df = data_rdd.toDF(["src", "dst"])

# Vertices dari kombinasi src dan dst
vertices_df = edges_df.select("src").union(edges_df.select("dst")).distinct().withColumnRenamed("src", "id")
```

```
# Tampilkan 10 edges pertama
edges_df.show(10)
```

```
+-----+
|  src|dst|
+-----+
| 53526| 1|
| 71632| 1|
| 98006| 1|
|148224| 1|
|209320| 1|
|268299| 1|
|270060| 1|
|302148| 1|
| 16283| 2|
| 37799| 2|
```

only showing top 10 rows

```
# Tampilkan 10 vertices pertama
vertices_df.show(10)
```

```
+-----+
|  id|
+-----+
|257784|
|156277|
|227553|
|123551|
|192348|
|327093|
|248828|
|108231|
|271864|
|227347|
```

only showing top 10 rows

Yang dilakukan:

- **Membentuk DataFrame edges_df**

Penjelasan:

- data_rdd awalnya adalah RDD berisi tuple pasangan produk: (produk_A, produk_B).
- toDF(["src", "dst"]) mengubahnya menjadi **DataFrame** dengan dua kolom:
 - src = produk asal (sumber),
 - dst = produk tujuan.

- **Membentuk DataFrame vertices_df**

Penjelasan:

- GraphFrame butuh daftar **simpul (vertices)** yang unik — semua produk yang muncul sebagai src atau dst.
- select("src") dan select("dst"): ambil semua produk dari kedua kolom.
- union(...): gabungkan keduanya jadi satu kolom.
- distinct(): hilangkan duplikat, karena satu produk bisa muncul berkali-kali.
- withColumnRenamed("src", "id"): ganti nama kolom jadi id (wajib untuk GraphFrame).

- **Menampilkan 10 baris pertama edges**

berfungsi untuk menampilkan **10 baris pertama** dari DataFrame edges_df, yang berisi pasangan produk yang sering dibeli bersama.

- Kolom src dan dst berisi ID produk,
- Formatnya sesuai untuk digunakan sebagai input dalam GraphFrame.

- **Menampilkan 10 baris pertama vertices**

menampilkan **10 simpul (vertex)** pertama dari DataFrame vertices_df, yaitu daftar unik semua ID produk yang muncul sebagai src atau dst dalam edges_df.

- Kolom sudah benar-benar bernama id (karena GraphFrame mengharuskan nama vertex ID = id).
- Tidak ada data yang kosong atau invalid (misalnya '' atau None).
- Vertex sudah benar-benar unik dan tidak duplikat.

4. Membuat Graph dan Menjalankan PageRank

```
# Buat GraphFrame
g = GraphFrame(vertices_df, edges_df)

# Jalankan PageRank
results = g.pageRank(resetProbability=0.15, maxIter=10)
```

Yang dilakukan:

- Membuat Objek GraphFrame

Membentuk graph dari dua DataFrame:

- vertices_df: berisi semua simpul (produk) unik.
- edges_df: berisi pasangan produk yang sering dibeli bersamaan (src dan dst).

- Menjalankan Algoritma PageRank

Penjelasan:

- resetProbability=0.15: parameter *damping factor* (biasanya 0.15 seperti pada PageRank klasik milik Google). Artinya, ada peluang 15% untuk melompat ke node acak — mencegah stuck di loop.
- maxIter=10: jumlah iterasi untuk menghitung nilai PageRank hingga konvergen (nilai stabil).

5. Hasil PageRank Urut dari Tertinggi

```
# Tampilkan 10 produk teratas berdasarkan PageRank
results.vertices.select("id", "pagerank") \
    .orderBy(col("pagerank").desc()) \
    .show(10, truncate=False)
```

```
+-----+-----+
|id|pagerank|
+-----+-----+
|27287|60.52335940468464|
|12994|48.50094460294497|
|6410|46.197265462121806|
|3958|42.230380031093105|
|1991|41.7966502506439|
|832|41.66109595285091|
|33532|41.20615917979095|
|4493|40.566737876513095|
|11097|40.15770403250736|
|23200|39.74231680370656|
+-----+-----+
only showing top 10 rows
```

Penjelasan detail:

- results.vertices: Mengakses DataFrame vertex hasil PageRank, yang berisi kolom id (produk) dan pagerank (skor PageRank).
- .select("id", "pagerank"): Memilih hanya dua kolom ini untuk ditampilkan.

- `.orderBy(col("pagerank").desc())`: Mengurutkan produk berdasarkan nilai pagerank secara menurun (dari yang paling tinggi ke rendah).
- `.show(10, truncate=False)`: Menampilkan 10 baris teratas secara lengkap (tidak memotong isi kolom).

Tujuan:

Menampilkan **10 produk yang paling berpengaruh** atau sentral di jaringan co-purchasing Amazon menurut skor PageRank. Produk-produk ini bisa jadi sangat populer, sering dibeli bersama banyak produk lain, atau menjadi "penghubung" antar kategori produk.